Memory Architecture and Cross-Run Learning

Design Principles

• Each agent owns its own memory. No agent reads another's private memory.

• Within-run memory contains only what that agent was allowed to see (filtered by information regime). The environment is the exception -- it keeps everything.

• Cross-run memory contains full information from completed past runs (the fog-of-war is lifted after a simulation ends).

• Agents summarize their own memory when context gets large. They never dump raw databases into LLM context -- they run analysis first, then include results.

• The user can truncate cross-run memory on request (old runs are archived, not deleted).

Memory Folder Structure

Per-Agent On-Disk Layout

Each agent (whether on its own machine or sharing one) maintains:

```
agent_data/
  {agent_id}/
    current_run/                    # (a) Within-run memory
      memory.db                     #   SQLite: decisions, observations, reflections
      dossier_snapshots/            #   Quarterly snapshots of own dossier
        Q1_2031.yaml
        Q2_2031.yaml
        ...
      analysis_cache/               #   Cached statistical analysis results
      gazette_archive/              #   Copies of industry gazettes received
      run_metadata.yaml             #   Run ID, seed, regime, start time

    across_runs/                    # (b) Cross-run memory (accumulated)
      runs_index.yaml               #   List of all past runs with metadata
      run_{run_id}/
        full_compustat.csv          #   Complete Compustat panel for that run
        full_debrief.csv            #   Complete debrief for that run
        my_decisions.csv            #   This agent's decisions with reasoning
        my_scores.csv               #   Scores received (financial + environment ratings)
        industry_gazettes/          #   All gazette issues from that run
        final_dossiers/             #   End-of-run dossiers for all firms
        run_summary.md              #   LLM-generated run summary (see below)
      summaries/
        policy_lessons.md           #   Accumulated lessons across runs
        historical_stats.db         #   SQLite: aggregated statistics for fast query
```

The Environment's Memory (Special Case)

The environment agent keeps everything -- it is the "historian":

```
agent_data/
  env_0/
    current_run/
      memory.db                     # Full world state each quarter
      all_dossiers/                 # Complete dossiers for every firm, every quarter
      all_firm_decisions/           # Every firm's raw decisions (for narrative generation)
      industry_ledger_history/      # Full ledger each quarter
      gazette_archive/              # Gazettes it authored
    across_runs/
      runs_index.yaml
      run_{run_id}/
        full_compustat.csv
        full_debrief.csv
        all_decisions/              # Every agent's decisions
        all_dossiers/               # Every firm's dossier history
        all_gazettes/
        all_scores.csv              # All scores including environment ratings
        run_narrative.md            # Full narrative of the run
      summaries/
        world_patterns.md           # Cross-run patterns in market dynamics
        historical_stats.db
```

The Environment Also Keeps Per-Player Archives

For analytical purposes and to provide player-specific historical context:

```
agent_data/
  env_0/
    player_archives/
      firm_0/
        run_{run_id}/
          decisions_and_outcomes.csv
          dossier_trajectory.yaml    # How the firm's dossier evolved
          score_card.yaml
      firm_1/
        ...
      ibank_0/
        ...
```

Within-Run Memory: What Each Agent Stores

Memory Database Schema (memory.db)

```
-- Decisions and reasoning (the core memory)
CREATE TABLE decisions (
    quarter INTEGER,
    phase TEXT,              -- 'ipo', 'quarterly', 'pricing', etc.
    decision_json TEXT,      -- the structured decision sent to orchestrator
```

```
    reasoning_trace TEXT,     -- full multi-step reasoning (reflect + strategize + decide)
    analysis_inputs TEXT,     -- JSON: what statistical analyses informed this decision
    confidence TEXT,          -- self-assessed: 'high', 'medium', 'low'
    created_at TIMESTAMP
);

-- What I observed from the world (filtered by info regime)
CREATE TABLE observations (
    quarter INTEGER,
    source TEXT,              -- 'orchestrator_context', 'sync_data', 'gazette'
    content_json TEXT,        -- structured observation data
    content_summary TEXT,     -- agent's own 1-paragraph summary (for later retrieval)
    created_at TIMESTAMP
);

-- Periodic self-assessments (every 4 quarters or on major events)
CREATE TABLE reflections (
    quarter INTEGER,
    trigger TEXT,             -- 'periodic', 'major_event', 'end_of_year', 'debrief'
    reflection TEXT,          -- LLM-generated strategic assessment
    key_metrics_json TEXT,    -- key performance indicators at time of reflection
    strategic_stance TEXT,    -- current strategic direction summary
    lessons_learned TEXT,     -- what worked, what didn't
    created_at TIMESTAMP
);

-- Statistical analysis results
CREATE TABLE analyses (
    quarter INTEGER,
    analysis_type TEXT,       -- 'trend', 'competitor', 'historical_comp', etc.
    query TEXT,               -- what was asked
    result_json TEXT,         -- structured result
    result_summary TEXT,      -- natural language summary
    created_at TIMESTAMP
);

-- Gazette summaries (the agent's own reading of the gazette)
CREATE TABLE gazette_notes (
    quarter INTEGER,
    gazette_text TEXT,        -- original gazette text
    my_interpretation TEXT,   -- agent's reading of what matters to them
    action_items TEXT,        -- what the agent thinks it should do about it
    created_at TIMESTAMP
);
```

What Goes Into Each Agent's memory.db

Agent Type	Observations Stored	Notes
Firm	Own private state, competitor publ	Only what the information regime a
Investment Bank	All firms' published financials, o	Does NOT store firms' private stat
Commercial Bank	All firms' published financials, o	Does NOT store firms' private stat
Credit Fund	Same as commercial bank	

Environment	Everything: all firm actions, all	Full information always

The Industry Gazette (Point 5)

What It Is

Every quarter, the environment agent produces an Industry Gazette -- a 1-2 page natural-language summary of what happened. It reads like a trade publication article.

Gazette Structure

```
gazette:
  issue: "SRT Industry Gazette -- Q2 2031 (Issue #2)"
  date: "July 2031"

  headline: "Aeterna Leads on Efficacy; GenVita Undercuts on Price"

  market_summary: |
    The SRT market grew to 4,200 active patients in Q2, up 12% from Q1.
    Total industry revenue reached $198M. Average treatment price declined
    slightly to $94,800 as GenVita's aggressive pricing pressured the market.

  firm_updates:
    - firm: "Aeterna Therapeutics"
      highlights: "Maintained premium pricing at $95K. Patient satisfaction highest
        in the industry at 7.8/10. Phase III enrollment reached 1,500 patients."
      concerns: "Third paralysis case reported. Local media coverage continues."

    - firm: "GenVita Sciences"
      highlights: "Captured largest market share (24.7%) with lowest price ($82K).
        Announced $120M capex for new facility in Singapore."
      concerns: "Gross margins thin at 42%. Cash burn rate unsustainable without
        additional financing."

    # ... for all active firms

  scientific_developments: |
    University of Tokyo published promising biomarker data for predicting
    paralysis susceptibility. If validated, this could reduce the most feared
    side effect to near-zero through patient screening.

  regulatory_update: |
    FDA held routine mid-cycle review of all ALT-approved products. No new
    safety signals identified. CMS announced a 6-month study on potential
    Medicare coverage for SRT therapies.

  market_outlook: |
    Analyst consensus: SRT market on track to reach $1B annual revenue by
    2033 if Gen 2 products arrive on schedule. Key risk: another paralysis
    cluster could stall adoption.
```

```
data_snapshot:
  total_patients: 4200
  total_revenue_quarterly: 198000000
  average_price: 94800
  industry_hhi: 2050
  total_serious_ae_ytd: 340
```

Gazette Generation

The environment agent generates the gazette as the LAST step of its quarterly reasoning pipeline (after determining outcomes and updating dossiers). It is prompted:

"Write a trade-publication-style summary of this quarter. Be factual and specific. Reference real numbers, real events, and real company names. This gazette will be read by all market participants. Maintain the tone of an informed industry observer, not a cheerleader or doomsayer."

Gazette Distribution

- The orchestrator stores the canonical gazette in the run outputs
- All agents receive the gazette via /sync
- Each agent stores the gazette and generates its own interpretation (gazette_notes table)
- The gazette archive is included in cross-run memory

Context Size Management (Point 6)

The Problem

Over 80 quarters, raw memory grows large:

- 80 decisions with full reasoning: ~200 KB
- 80 observations: ~400 KB
- 80 gazettes: ~160 KB
- 80 analysis results: ~120 KB

Total: ~1 MB of text -- far too much for any LLM context window.

The Solution: Agent-Side Summarization

Agents manage their own context. They NEVER upload raw databases to the LLM. Instead:

Short-term memory (last K quarters, K=4 by default): Full detail. All decisions, observations, analyses, and gazettes for the most recent 4 quarters are included verbatim in the prompt.

Medium-term memory (quarters K+1 to 3K): Summarized. The agent runs a periodic summarization task (every 4 quarters):

- LLM call: "Summarize your decisions and outcomes for quarters [N-4] to [N-1].

Focus on: key strategic choices, their outcomes, lessons learned, and unresolved questions. Compress to 1-2 paragraphs."

- The summary replaces the raw entries in the prompt (raw data stays in SQLite).

Long-term memory (quarters > 3K): Highly compressed. The agent maintains a rolling "strategic narrative":

- "Over the first year, I pursued a premium pricing strategy, investing heavily

in R&D ($120M). This resulted in [outcomes]. Key lesson: [lesson]."

- Updated every 8 quarters by re-summarizing medium-term summaries.
- Maximum 2 paragraphs in the prompt.

The Summarization Trigger

```
def prepare_context_for_prompt(agent, current_quarter):
    """Build the memory section of the prompt."""

    # Short-term: full detail
    short_term = agent.memory.get_entries(
        quarter_min=current_quarter - 4,
        quarter_max=current_quarter - 1,
        full_detail=True
    )

    # Medium-term: check if summarization needed
    medium_start = max(1, current_quarter - 12)
    medium_end = current_quarter - 5
    if medium_start <= medium_end:
        summary = agent.memory.get_summary(medium_start, medium_end)
        if summary is None or summary.is_stale:
            # Agent summarizes its own memory (LLM call)
            raw_entries = agent.memory.get_entries(medium_start, medium_end)
            summary = agent.summarize_memory(raw_entries)
            agent.memory.store_summary(medium_start, medium_end, summary)

    # Long-term: rolling narrative
    if current_quarter > 12:
        long_term = agent.memory.get_long_term_narrative()
        if long_term is None or long_term.is_stale:
            all_summaries = agent.memory.get_all_summaries()
            long_term = agent.compress_to_narrative(all_summaries)
            agent.memory.store_long_term_narrative(long_term)

    return {
        "short_term": short_term,       # ~20-40 KB
        "medium_term": summary,          # ~2-4 KB
        "long_term": long_term,          # ~1-2 KB
        "cross_run": cross_run_summary   # ~2-4 KB (see below)
    }
```

Analysis Instead of Data Upload

When an agent needs to understand trends in its data, it does NOT put the data in the LLM prompt. Instead:

```
1. Agent's brain.py identifies a question: "Is my market share declining?"

2. Agent's analyst.py runs the query on SQLite:
   SELECT quarter, market_share FROM observations WHERE quarter > 60
   -> Returns: [(61, 0.22), (62, 0.21), (63, 0.19), (64, 0.18)]
```

```
3. Agent's analyst.py computes statistics:
   trend_analysis([0.22, 0.21, 0.19, 0.18]) -> {
     "direction": "declining",
     "avg_quarterly_change": -0.013,
     "is_significant": true
   }

4. The RESULT (not the raw data) goes into the LLM prompt:
   "Your market share has declined from 22% to 18% over the last 4 quarters,
    averaging -1.3 points per quarter. This decline is statistically significant."

5. The LLM reasons about the interpreted result, not raw numbers.
```

Cross-Run Memory in Prompts

Similarly, past simulation data is never dumped raw. Instead:

```
1. Agent queries across_runs/historical_stats.db:
   "Find firms similar to me (Gen 1, ~$300M assets, ~20% share, heavy R&D)"

2. Returns 5 comparable firm-quarters from past runs

3. Summarized for prompt:
   "In 3 past simulations, firms in your position that invested >$30M/quarter
    in product R&D achieved Gen 2 within 12 quarters (median). Those that
    cut R&D below $15M/quarter never advanced and lost significant market share.
    Average equity IRR for the heavy-R&D group was 28% annualized."
```

Cross-Run Memory Dispatch (Point 2)

End-of-Run Process

When a simulation completes:

```
1. ORCHESTRATOR compiles final run package:
   - Complete Compustat panel
   - Complete debrief with all scores (financial + environment ratings)
   - All industry gazettes
   - Final dossiers for all firms
   - Run metadata (seed, regimes, parameters, duration)

2. ORCHESTRATOR dispatches to EACH agent via POST /archive:
   Agent receives:
   - The full Compustat panel (now unfiltered -- fog of war lifted)
   - The full debrief
   - Their own complete decision history + reasoning traces
   - Their scores
   - All gazettes
   - All final dossiers
   - A "run summary" prompt asking them to reflect:
     "The simulation has ended. Here is the complete history.
      Write a 2-paragraph summary of what happened and what you learned."
```

```
3. EACH AGENT stores this in across_runs/run_{run_id}/:
   - Archives all received data
   - Generates run_summary.md (LLM call)
   - Updates summaries/policy_lessons.md:
     "Based on this new run, update your accumulated lessons.
      What strategies worked? What didn't? What would you do differently?"
   - Updates summaries/historical_stats.db with aggregated statistics

4. ENVIRONMENT AGENT additionally:
   - Stores the per-player archives
   - Generates a full run narrative (run_narrative.md)
   - Updates world_patterns.md with cross-run market dynamics
```

User-Initiated Truncation

The user can request truncation of old cross-run memory:

```
# Archive runs older than run_id 5 (moves to cold storage, removes from active memory)
python -m llm_firm_lab memory truncate --keep-last 5

# Or by date
python -m llm_firm_lab memory truncate --keep-since 2026-04-01

# Or selectively
python -m llm_firm_lab memory truncate --remove-runs run_001,run_002

# View current memory size
python -m llm_firm_lab memory status
```

Truncation does NOT delete data. It moves old runs from across_runs/ to across_runs/archived/ and removes them from the active runs_index.yaml. The agent's historical_stats.db and policy_lessons.md retain the aggregated learning even after the raw data is archived.

How Cross-Run Memory Grows

Runs Completed	Active across_runs/ Size (per agen	Notes
1	~5 MB	One full run
5	~25 MB	Manageable
20	~100 MB	Consider truncating older runs
50+	~250 MB+	Should truncate; aggregated stats

The historical_stats.db (aggregated) stays small (~1 MB) regardless of how many runs are archived.

Memory Isolation Guarantees

Within a Run

Agent	Can Access Own Memory	Can Access Others' Memory	Can Access Raw Data
Any firm	Yes (filtered by info reg	No	Own SQLite only
Any financial inst.	Yes (filtered by info reg	No	Own SQLite only
Environment	Yes (full)	Yes (read-only, for narra	All data

Orchestrator	N/A (not an agent)	N/A	Everything (it is the sou

Across Runs

After a run completes and memory is dispatched, ALL agents receive the FULL unfiltered data from the completed run. The information regime only applies during the run.

Rationale: Past runs are "history" -- the agents should learn from the complete picture, not from their limited wartime perspective. A firm agent that was running blind during a simulation should be able to look back and see "oh, my competitor had 2x my R&D budget -- that's why they beat me to Gen 2."

Configuration

```
memory:
  short_term_quarters: 4           # full detail in prompt
  medium_term_quarters: 12         # summarized in prompt
  summarization_interval: 4        # re-summarize every N quarters
  max_prompt_memory_tokens: 8000   # hard cap on memory section of prompt
  cross_run_max_active: 10         # max past runs in active memory
  cross_run_retrieval_k: 5         # top-K similar cases from past runs

  # Gazette
  gazette_max_length_words: 800
  gazette_save_to_disk: true

  # Context management
  agent_self_summarize: true       # agents manage their own context
  fallback_truncation: "oldest_first"  # if still too large after summarization
```